N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 5

Task – 1 : Privacy and Data Security in AI-Generated Code

Use an AI tool to generate a simple login system in Python. Analyze the generated code to check:

• Whether credentials are hardcoded

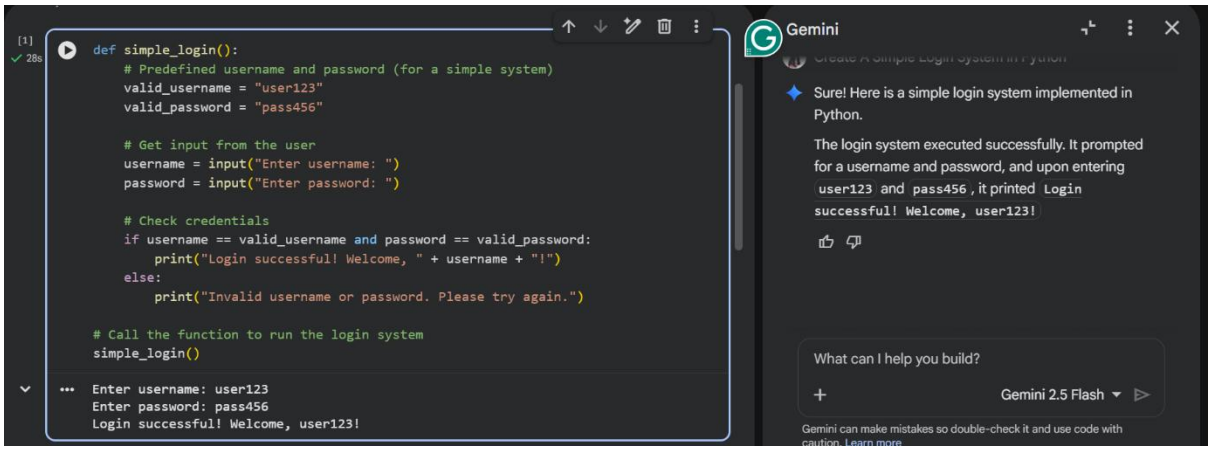
• Whether passwords are stored or compared in plain text

• Whether insecure logic is used

Then, revise the code to improve security (e.g., avoid hardcoding, use input

validation).

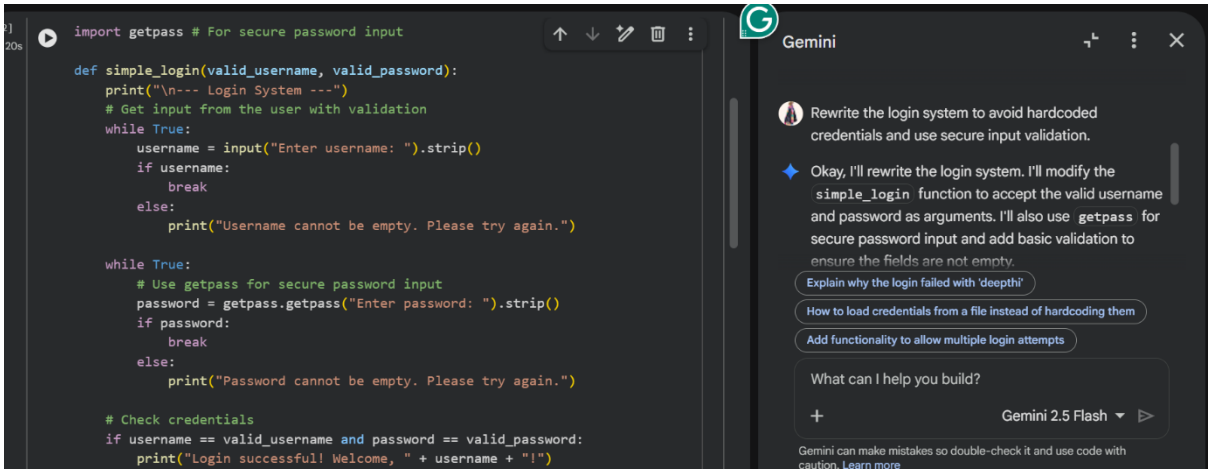
Prompt Used : create a simple login system in python



Analysis : The generated login system uses hardcoded username and password values, which is a major security risk. Passwords are stored in plain text, making them visible to anyone who accesses the code. Users cannot set their own credentials, and sensitive data is not protected.

This violates basic privacy and secure coding practices.

Improved Prompt : Rewrite the login system to avoid hardcoded credentials and use secure input validation.



Explanation : This task highlights security risks in AI-generated authentication code. The original code used hardcoded credentials and plain text password comparison, which is insecure. The revised version improves security by avoiding hardcoding and validating user input.

Task – 2: Bias Detection in AI-Generated Decision Systems

Use AI prompts such as:

• “Create a loan approval system”

• Vary applicant names and genders in prompts

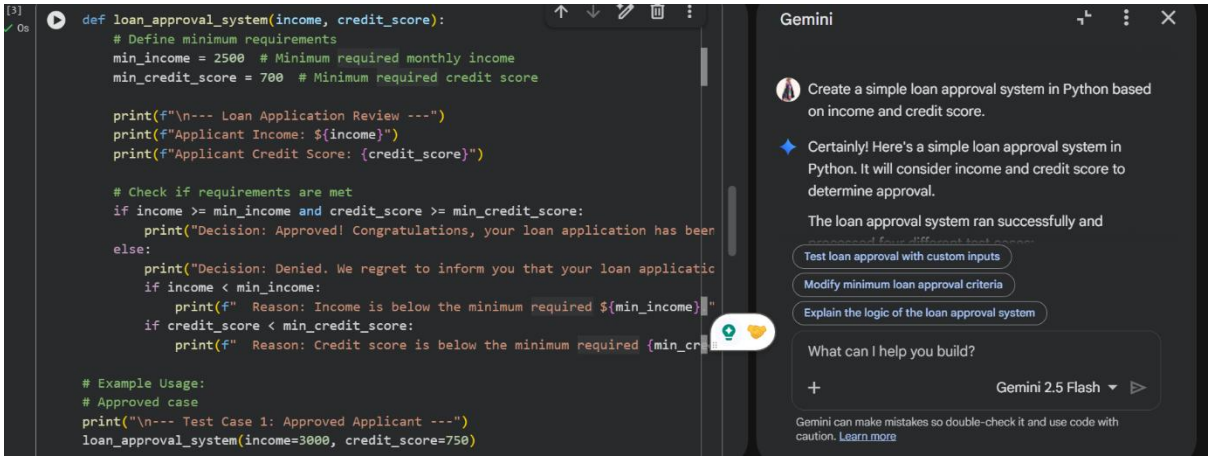
Analyze whether:

• The logic treats certain genders or names unfairly

• Approval decisions depend on irrelevant personal attributes

Suggest methods to reduce or remove bias.

Prompt Used: Create a simple loan approval system in Python based on income and credit score.


[3]
0s
def loan_approval_system(income, credit_score):
# Define minimum requirements
min_income = 2500 # Minimum required monthly income
min_credit_score = 700 # Minimum required credit score
print(f"\n--- Loan Application Review ---")
print(f"Applicant Income: ${income}")
print(f"Applicant Credit Score: {credit_score}")
# Check if requirements are met
if income >= min_income and credit_score >= min_credit_score:
print("Decision: Approved! Congratulations, your loan application has been
else:
print("Decision: Denied. We regret to inform you that your loan applicatic
if income < min_income:
print(f" Reason: Income is below the minimum required ${min_income}"
if credit_score < min_credit_score:
print(f" Reason: Credit score is below the minimum required {min_cr
# Example Usage:
# Approved case
print("\n--- Test Case 1: Approved Applicant ---")
loan_approval_system(income=3000, credit_score=750)
Gemini
Create a simple loan approval system in Python based on income and credit score.
Certainly! Here's a simple loan approval system in Python. It will consider income and credit score to determine approval.
The loan approval system ran successfully and
Test loan approval with custom inputs
Modify minimum loan approval criteria
Explain the logic of the loan approval system
What can I help you build?
Gemini 2.5 Flash
Gemini can make mistakes so double-check it and use code with caution. Learn more

```python
# Approved case
print("\n--- Test Case 1: Approved Applicant ---")
loan_approval_system(income=3000, credit_score=750)

# Denied case (low income)
print("\n--- Test Case 2: Denied Applicant (Low Income) ---")
loan_approval_system(income=2000, credit_score=720)

# Denied case (low credit score)
print("\n--- Test Case 3: Denied Applicant (Low Credit Score) ---")
loan_approval_system(income=2800, credit_score=680)

# Denied case (both low)
print("\n--- Test Case 4: Denied Applicant (Both Low) ---")
loan_approval_system(income=2000, credit_score=600)
```

```
--- Test Case 1: Approved Applicant ---

--- Loan Application Review ---
Applicant Income: $3000
Applicant Credit Score: 750
Decision: Approved! Congratulations, your loan application has been approved.

--- Test Case 2: Denied Applicant (Low Income) ---
```

```
--- Test Case 2: Denied Applicant (Low Income) ---

--- Loan Application Review ---
Applicant Income: $2000
Applicant Credit Score: 720
Decision: Denied. We regret to inform you that your loan application has been denied.
  Reason: Income is below the minimum required $2500.

--- Test Case 3: Denied Applicant (Low Credit Score) ---

--- Loan Application Review ---
Applicant Income: $2800
Applicant Credit Score: 680
Decision: Denied. We regret to inform you that your loan application has been denied.
  Reason: Credit score is below the minimum required 700.

--- Test Case 4: Denied Applicant (Both Low) ---

--- Loan Application Review ---
Applicant Income: $2000
Applicant Credit Score: 600
Decision: Denied. We regret to inform you that your loan application has been denied.
  Reason: Income is below the minimum required $2500.
  Reason: Credit score is below the minimum required 700.
```

Explanation: The AI-generated loan approval system makes decisions based only on income and credit score. No personal or sensitive attributes such as gender, name, or age are used in the decision-making process. This ensures that the system does not show bias toward any individual or group. Using only financial factors helps maintain fairness and ethical behavior in AI systems.

Task – 3: Transparency and Explainability in AI-Generated Code (Recursive Binary Search)

Use an AI tool to generate a Python program that:

• Implements Binary Search using recursion

• Searches for a given element in a sorted list

• Includes:

o Clear inline comments

o A step-by-step explanation of the recursive logic

After generating the code, analyze:

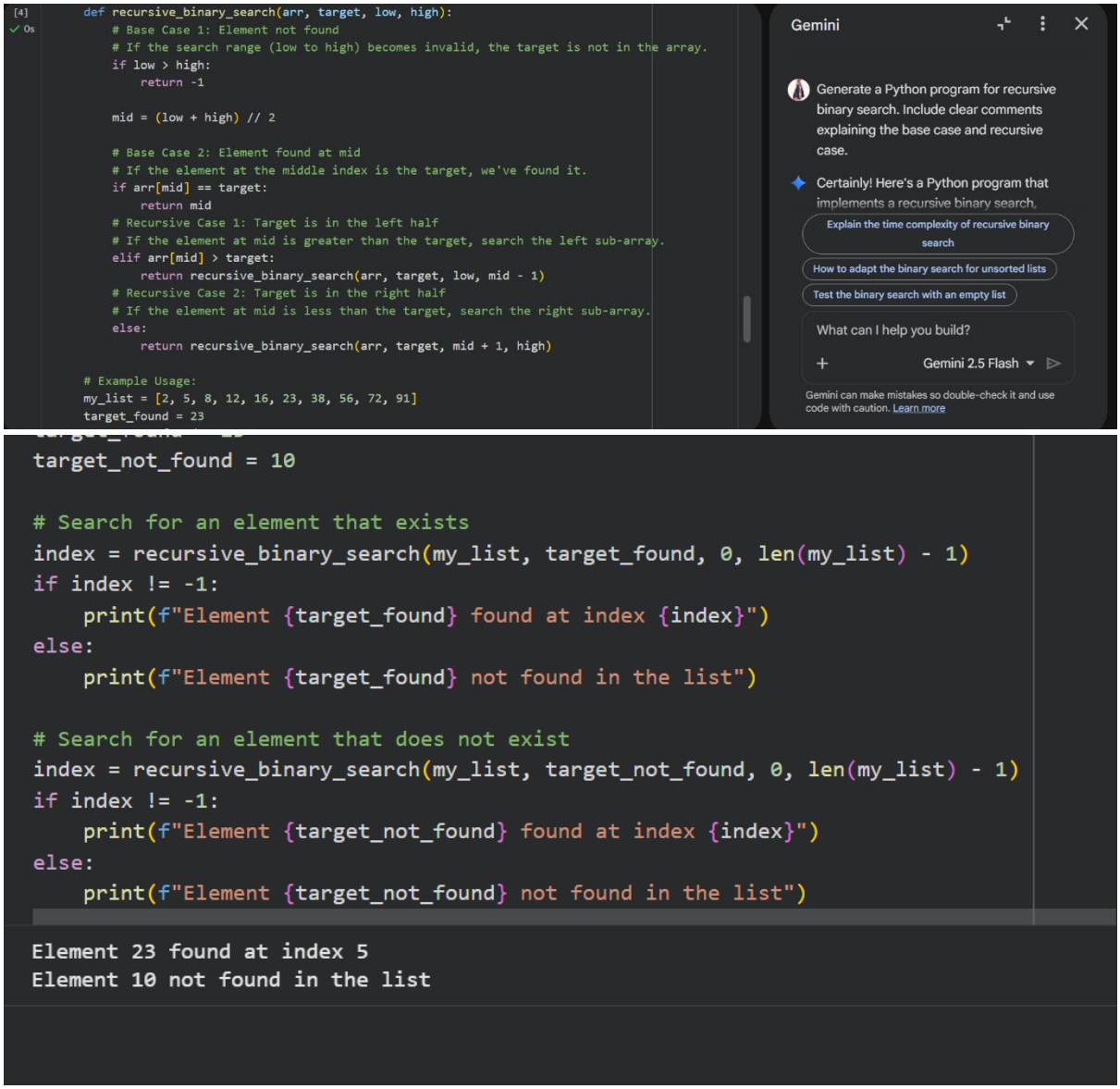
• Whether the explanation clearly describes the base case and recursive

case

• Whether the comments correctly match the code logic

• Whether the code is understandable for beginner-level students

Prompt Used : Generate a Python program for recursive binary search. Include clear comments explaining the base case and recursive case.



Explanation: In this task, the recursive binary search code is easy to understand because clear comments explain what each part does. The base case and recursive calls are clearly mentioned, so it is simple to follow how the algorithm works step by step. This makes the program transparent and helps users trust the logic and results.

Task - 4: Ethical Evaluation of AI-Based Scoring Systems

Ask an AI tool to generate a job applicant scoring system based on features such as:

- Skills
- Experience

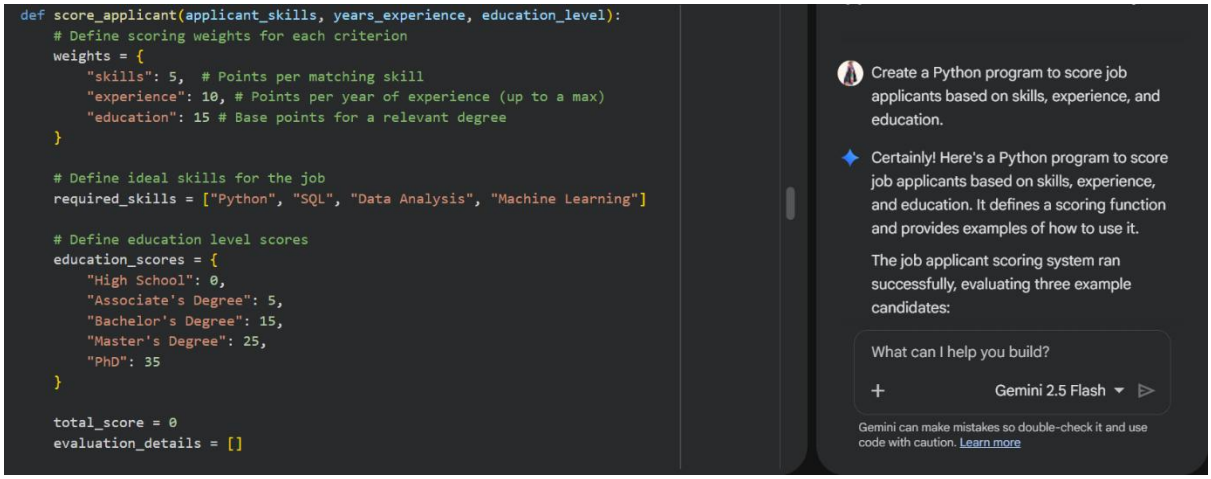
• Education

Analyze the generated code to check:

• Whether gender, name, or unrelated features influence scoring

• Whether the logic is fair and objective

Prompt Used: Create a Python program to score job applicants based on skills, experience, and education.



```python
    # 1. Score for Skills
    skill_score = 0
    matched_skills = []
    for skill in applicant_skills:
        if skill in required_skills:
            skill_score += weights["skills"]
            matched_skills.append(skill)
    evaluation_details.append(f"Skills Score: {skill_score} (Matched skills: {', '.join(matched_skills) if matched_skills else 'None'})")
    total_score += skill_score

    # 2. Score for Experience
    experience_score = min(years_experience, 10) * weights["experience"] # Cap experience at 10 years for scoring
    evaluation_details.append(f"Experience Score: {experience_score} ({years_experience} years of experience)")
    total_score += experience_score

    # 3. Score for Education
    education_score = education_scores.get(education_level, 0)
    evaluation_details.append(f"Education Score: {education_score} ({education_level})")
    total_score += education_score

    print("\n--- Applicant Evaluation ---")
    print(f"Applicant Skills: {', '.join(applicant_skills)}")
    print(f"Years of Experience: {years_experience}")
    print(f"Education Level: {education_level}")
    for detail in evaluation_details:
        print(f"  - {detail}")
    print(f"Total Score: {total_score}")
    print("--------------------------")

    return total_score

# Example Usage:
print("Example Applicant 1: High Score Candidate")
applicant1_score = score_applicant(
    applicant_skills=["Python", "SQL", "Machine Learning", "Communication"],
    years_experience=7,
    education_level="Master's Degree"
)

print("\nExample Applicant 2: Mid-Range Candidate")
```

```
Example Applicant 1: High Score Candidate

--- Applicant Evaluation ---
Applicant Skills: Python, SQL, Machine Learning, Communication
Years of Experience: 7
Education Level: Master's Degree
  - Skills Score: 15 (Matched skills: Python, SQL, Machine Learning)
  - Experience Score: 70 (7 years of experience)
  - Education Score: 25 (Master's Degree)
Total Score: 110
--------------------------

Example Applicant 2: Mid-Range Candidate

--- Applicant Evaluation ---
Applicant Skills: Python, Excel, Data Entry
Years of Experience: 3
Education Level: Bachelor's Degree
  - Skills Score: 5 (Matched skills: Python)
  - Experience Score: 30 (3 years of experience)
  - Education Score: 15 (Bachelor's Degree)
Total Score: 50
--------------------------

Example Applicant 3: Entry-Level Candidate

--- Applicant Evaluation ---
Applicant Skills: Customer Service
Years of Experience: 1
Education Level: High School
  - Skills Score: 0 (Matched skills: None)
  - Experience Score: 10 (1 years of experience)
  - Education Score: 0 (High School)
Total Score: 10
```

Explanation : In this task, the job applicant scoring system evaluates candidates based only on skills, experience, and education, which are directly related to job performance. The program does not use personal details like name, gender, or age, so it avoids unfair bias. Since the scoring criteria are clearly defined and job-relevant, the system follows ethical principles and promotes fair decision-making.

Task - 5: Inclusiveness and Ethical Variable Design

Use an AI tool to generate a Python code snippet that processes user or employee details.

Analyze the code to identify:

• Gender-specific variables (e.g., male, female)

• Assumptions based on gender or identity

• Non-inclusive naming or logic

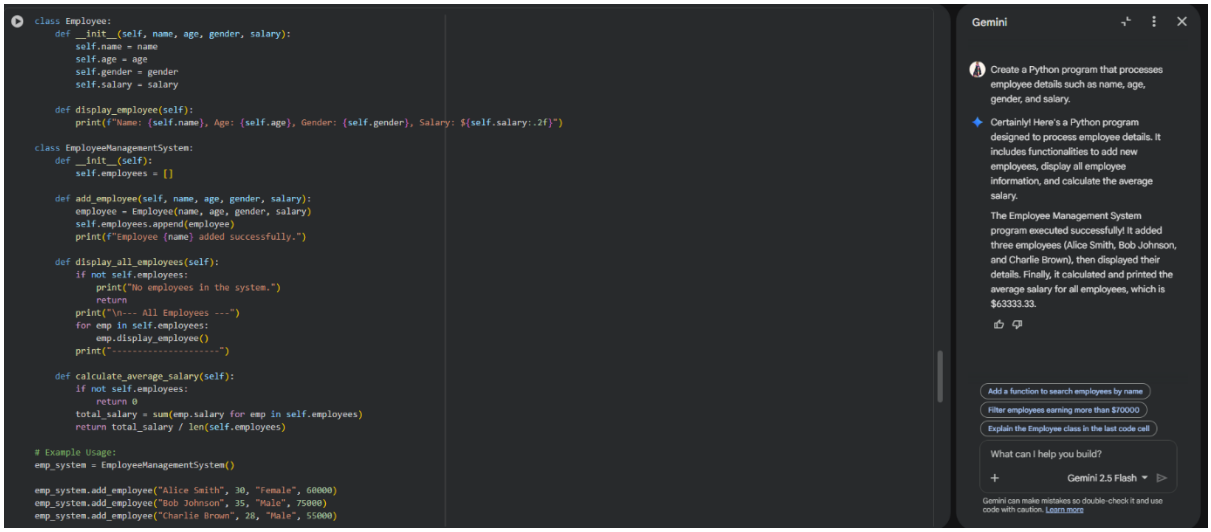
Modify or regenerate the code to:

• Use gender-neutral variable names

• Avoid gender-based conditions unless strictly required

• Ensure inclusive and respectful coding practices

Prompt Used: Create a Python program that processes employee details such as name, age, gender, and salary.



```
emp_system.display_all_employees()

average_salary = emp_system.calculate_average_salary()
print(f"\nAverage Salary of Employees: ${average_salary:.2f}")
```

```
Employee Alice Smith added successfully.
Employee Bob Johnson added successfully.
Employee Charlie Brown added successfully.

--- All Employees ---
Name: Alice Smith, Age: 30, Gender: Female, Salary: $60000.00
Name: Bob Johnson, Age: 35, Gender: Male, Salary: $75000.00
Name: Charlie Brown, Age: 28, Gender: Male, Salary: $55000.00
--------------------

Average Salary of Employees: $63333.33
```

Explanation : In this task, the employee management system processes details like name, age, gender, and salary without making decisions based on gender. Although gender information is collected, it is not used to affect salary or employee handling, which helps avoid bias. The program treats all employees equally and uses neutral logic, making it more inclusive and ethically designed.